Generative AI Academy

LLM INTERNALS

HIGH LEVEL STEPS IN TRANSFORMER INFERENCING

Input tokens

↓

Embedding

↓

Transformer layers

↓

[$QK^T \rightarrow$ weights $\rightarrow$ weights × V]

↓

Final hidden vector (h)

↓

Unembedding (h × W_unembed)

↓

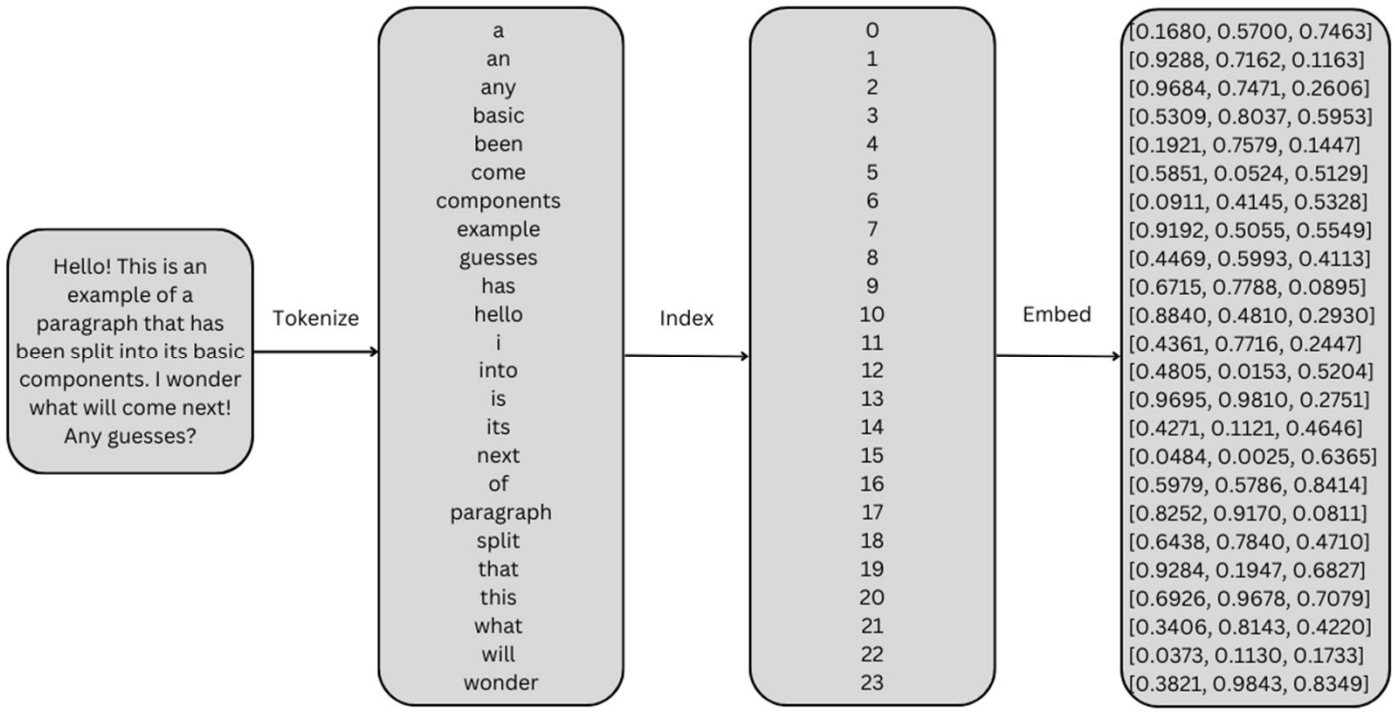
Softmax

↓

Next token prediction

KEY CONCEPT – SELF ATTENTION

Step 1: token -> embedding (Embedding Layer)



Size: (seq_length, d_model).

Traditional models struggle with long range dependencies

Example:

“Server failed because power supply overheated”
→ “failed” is associated with “power supply”

Need a way to focus on relevant parts

Step 2: Contextualize the embeddings

Embeddings will be contextualized using the attention layer

See streamlit demo for visualizetion

Size: (seq_length, d_model).

Attention Mechanism

How is the word associated with other words in the context?

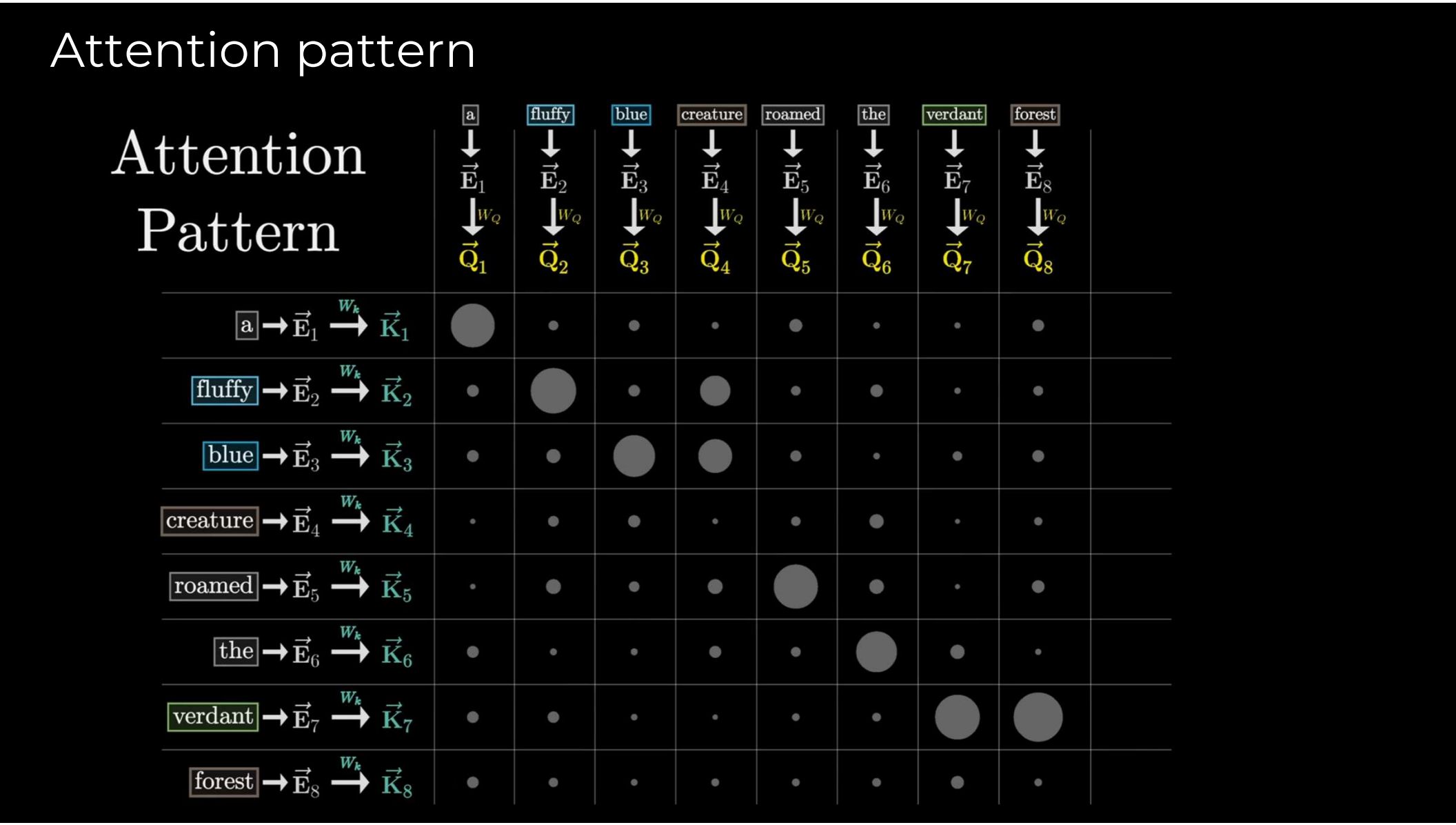
Q = What am I looking for
K = What do I have?
V = Value of the token

Sentence:
“Fan failure caused server shutdown”

“shutdown” attends strongly to:
“failure”
“fan”

Output embeds **context awareness**

Attention pattern



Example

Sentence: "The server shutdown due to overheating“

For token "shutdown":

Query asks: “What caused me?

”Keys:
"server" → medium match
"overheating“ → high match

Dot product gives:

shutdown · overheating → high score

Why dot product?
Because it measures: Similarity / alignment
High value → strong relationship

Demo

https://resources.criodo.com/resources/attention-mechanism-math/

Streamlit app

Step 2: Weighted sums

This is a classic neural network operation:

X = input embeddings (tokens)
W = learned weights

Q = X × Wq
K = X × Wk
V = X × Wv

Each output value = weighted sum of input features

✔ This is exactly like: Dense layerLinear transformation

Step 3: Dot product similarity computation: Q x K(t)

Intuition

Think of it like:
Q (Query) = “What am I looking for?”
K (Key) = “What do I contain?”

Multiplying them answers:

“How relevant is token i to token j?”

For a single pair:

```
score(i, j) = Q[i] · K[j]
```

Why dot product?
Because it measures: Similarity / alignment
High value → strong relationship

Full Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) \times V$$

Step 1: Compute similarity:

$Q \times K^T \rightarrow$ scores

Step 2: Normalize (softmax)

weights = softmax(scores)

Now these are: Probabilities, Sum to 1

Step 3: Apply to V (THIS is weighted sum)

Output = weights × V (added for all vectors in the context)

Key insight

👉 `Q × K^T` → computes weights

👉 `weights × V` → applies weighted sum

Full example

In the sentence Shutdown caused by overheating in power unit
For token “shutdown”

Q: What I am I looking for? : “What caused the shutdown?”

Step 1: Compare with all words (Keys)

Word	Match score
server	0.3
overheating	0.9
power	0.5

This is: $Q \times K^T$

Step 2: Normalize (softmax)

Word	Weight
server	0.2
Overheating	0.6
power	0.2

Step 3: Combine meanings (Values)
Output = 0.2·V(server) + 0.6·V(overheating) + 0.2·V(power)
👉 THIS is the final weighted sum

Positional Encoding

Transformer sees tokens in parallel
No inherent order awareness

But order is important:

Sentence:
“Server failed after reboot”

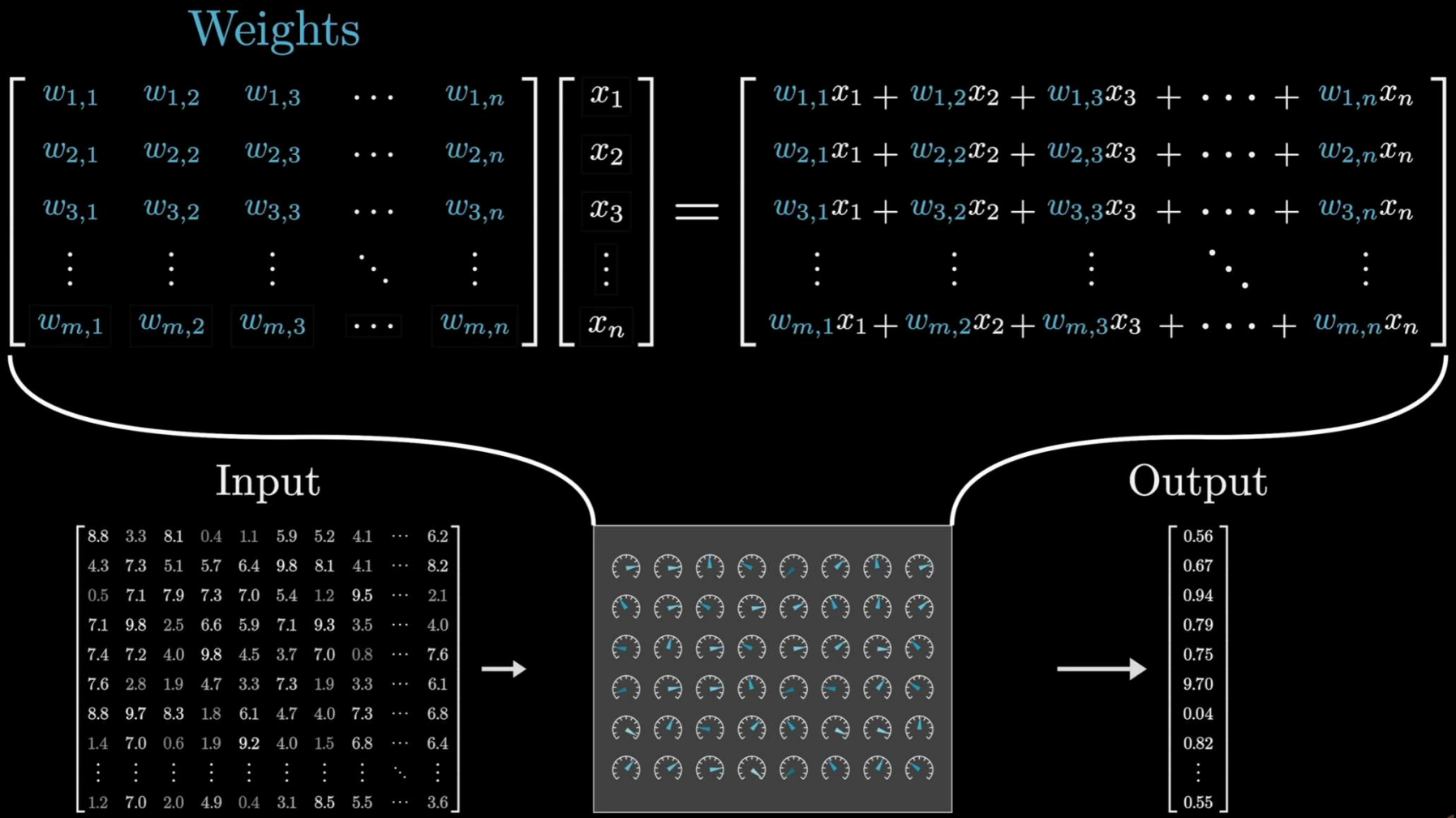
“Reboot failed after server upgraded”

Meaning changes!!

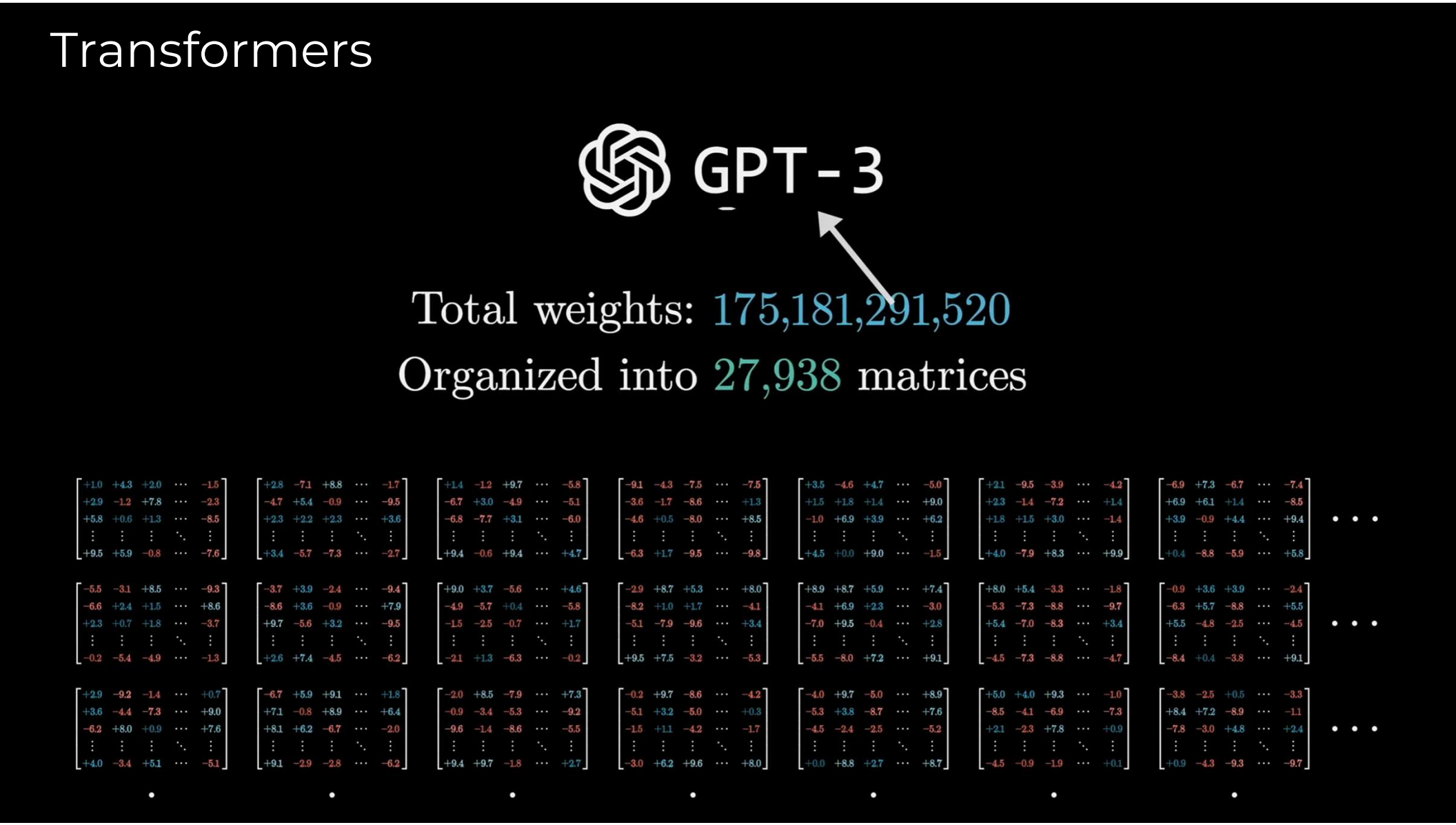
Sinusoidal encodings are used to embed position into the vector

- Generalizes to longer sequences
- Lean relative distances

Transformers



Transformers



Transformers

GPT-3

Total weights:

175,181,291,520

Embedding	$\overset{12,288}{d\_embed} * \overset{50,257}{n\_vocab}$	$= 617,558,016$
Key	$\overset{128}{d\_query} * \overset{12,288}{d\_embed} * \overset{96}{n\_heads} * \overset{96}{n\_layers}$	$= 14,495,514,624$
Query	$\overset{128}{d\_query} * \overset{12,288}{d\_embed} * \overset{96}{n\_heads} * \overset{96}{n\_layers}$	$= 14,495,514,624$
Value	$\overset{128}{d\_value} * \overset{12,288}{d\_embed} * \overset{96}{n\_heads} * \overset{96}{n\_layers}$	$= 14,495,514,624$
Output	$\overset{12,288}{d\_embed} * \overset{128}{d\_value} * \overset{96}{n\_heads} * \overset{96}{n\_layers}$	$= 14,495,514,624$
Up-projection	$\overset{49,152}{n\_neurons} * \overset{12,288}{d\_embed} * \overset{96}{n\_layers}$	$= 57,982,058,496$
Down-projection	$\overset{12,288}{d\_embed} * \overset{49,152}{n\_neurons} * \overset{96}{n\_layers}$	$= 57,982,058,496$
Unembedding	$\overset{50,257}{n\_vocab} * \overset{12,288}{d\_embed}$	$= 617,558,016$

Transformers

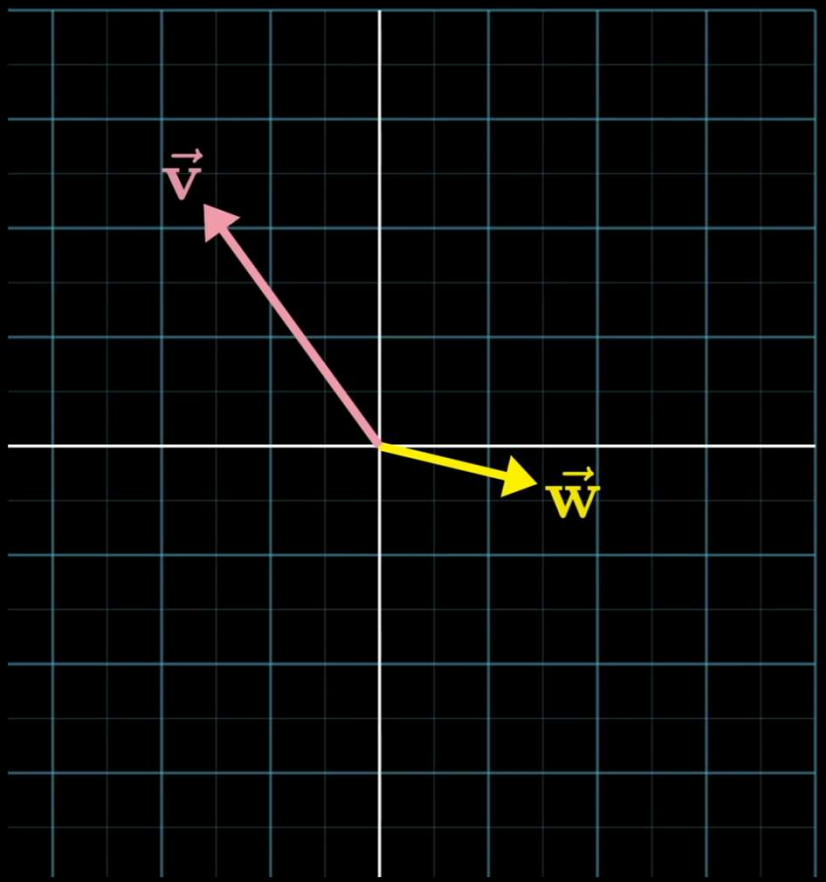
Total weights: 175,181,291,520

Organized into 27,938 matrices

GPT-3

Embedding	
Key	
Query	
Value	
Output	
Up-projection	
Down-projection	
Unembedding	

Importance of dot products of 2 vectors

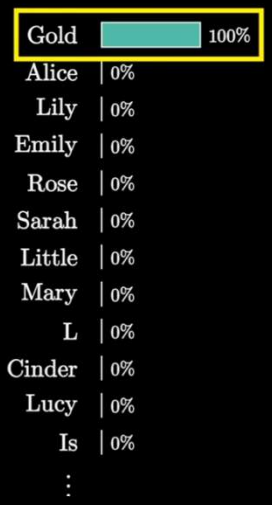


$$\underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_n \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_n \end{bmatrix}}_{\text{Dot product}} = \begin{matrix} v_1 w_1 \\ + \\ v_2 w_2 \\ + \\ v_3 w_3 \\ + \\ \vdots \\ + \\ v_n w_n \\ \| \\ -3.11 \end{matrix}$$

Transformers

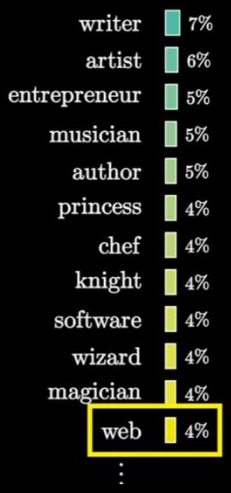
Temp = 0

Once upon a time, there was a little girl named Gold

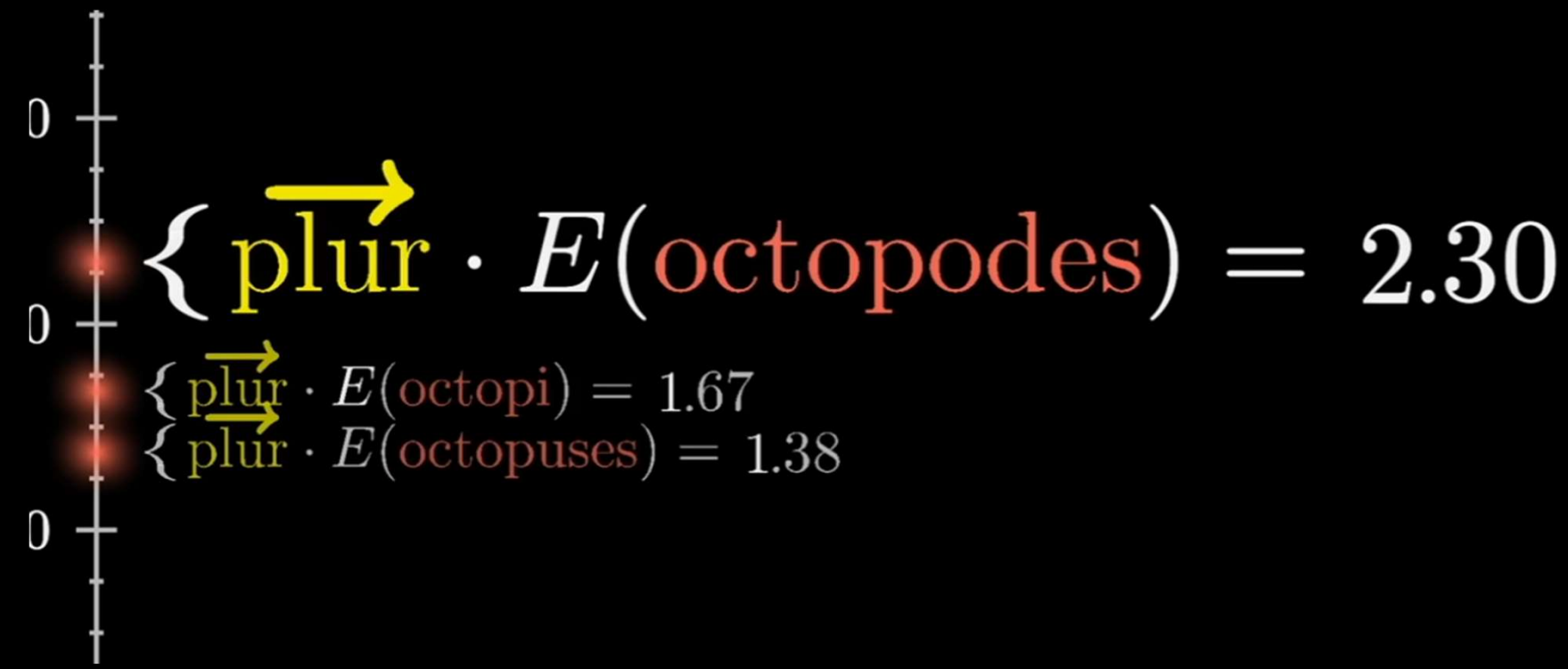


Temp = 5

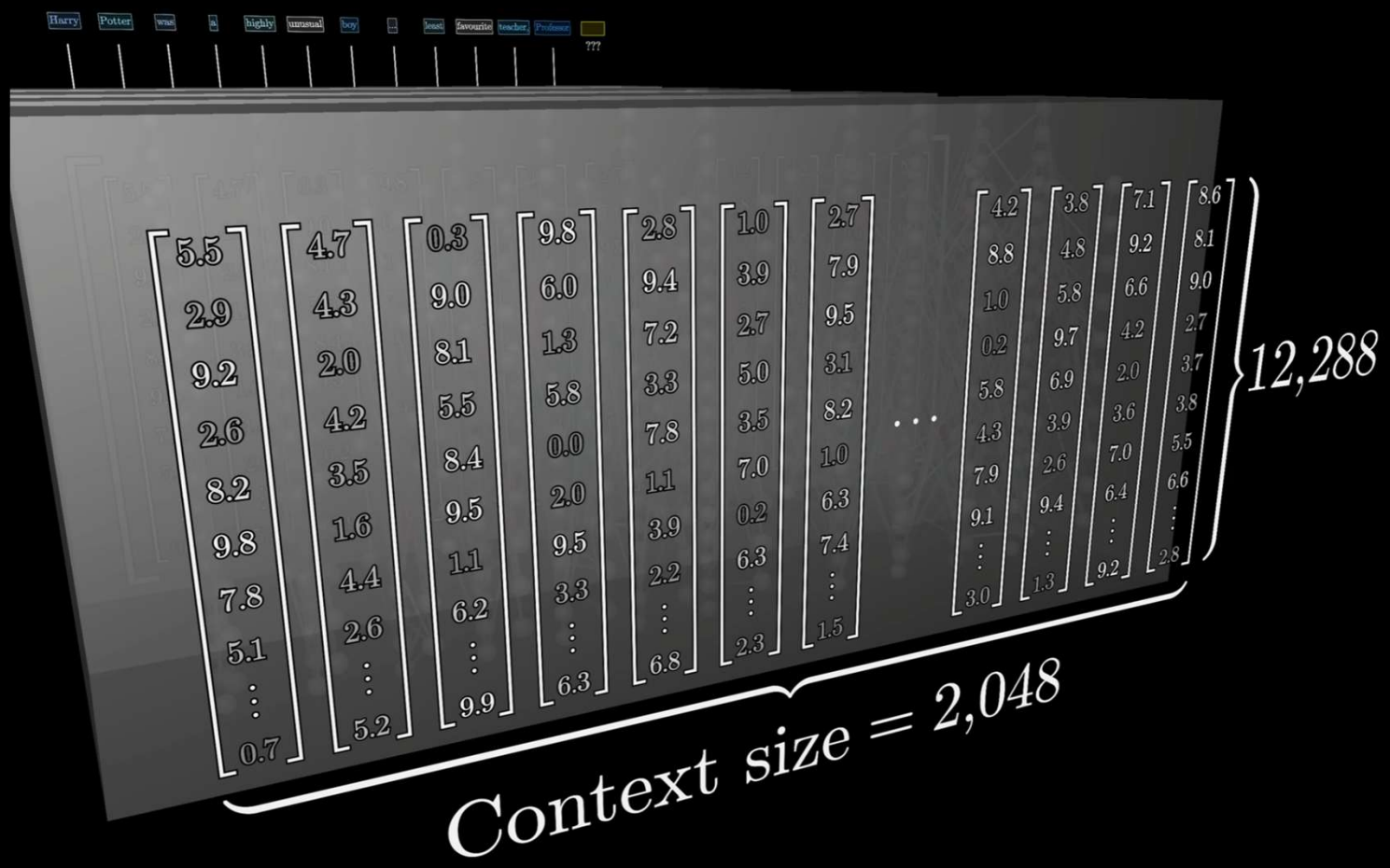
Once upon a time, there was a young and aspiring web



Importance of dot products of 2 vectors



Context size and number of dimensions



System Design considerations for LLM Workloads

Matrix Multiply (GEMM) as the Core of Attention

Why everything reduces to GEMM

At inference time, Transformers are basically massive matrix multiplications:
Core operations in attention:

$Q = X \cdot Wq$
$K = X \cdot Wk$
$V = X \cdot Wv$

Attention scores = $Q \cdot K^T$
Output = $\text{softmax}(QK^T) \cdot V$

These are all GEMM (General Matrix Multiply) operations.
Why GPUs excel here (Tensor Cores)
Modern GPUs (NVIDIA H100, A100):Use Tensor Cores optimized for:
FP16 / BF16 / FP8Matrix tiles (e.g., 16×16, 64×64 blocks)
Instead of doing scalar multiply-add:
C[i][j] += A[i][k] * B[k][j]
Tensor cores do:
C_tile = A_tile × B_tile
Thousands of multiply-adds in one clock

Matrix Multiply (GEMM) as the Core of Attention

Transformer Operation	GPU Mapping
Q, K, V projection	GEMM on tensor cores
$Q \times K^T$	GEMM
softmax	element-wise (less compute-heavy)
output projection	GEMM

💡 ~90%+ of compute time = GEMM

Key sizing implication for HPE

Model throughput depends on:
Tensor core TFLOPS
Precision (FP8 > BF16 > FP32)
H100 FP8 → massive throughput improvement
Rule:
If workload = LLM inference → optimize for Tensor Core throughput, not CPU cores.

SIMD vs SIMT — Why GPUs dominate CPUs

CPU = SIMD (Single Instruction, Multiple Data)

- Few cores (8–64)
- Vector instructions (AVX-512)
- Good for:
  - Branching logic
  - Sequential workloads

GPU = SIMT (Single Instruction, Multiple Threads)

- Thousands of lightweight threads
- Warps (32 threads execute same instruction)

```
Warp executes:
ADD r1, r2, r3  → across 32 threads simultaneously
```

Why SIMT wins for inference

Transformer workloads:

- Highly parallel
- Same operation on many tokens / neurons

Example:

- Multiply 4096×4096 matrices → perfectly parallel

CPU vs GPU reality

Metric	CPU	GPU
Cores	~32	10,000+ threads
Parallelism	Limited	Massive
Throughput	Low	Extremely high
Latency (single task)	Better	Worse

Key insight for HPE sizing

- CPUs:
  - Orchestration
  - Tokenization
  - API handling
- GPUs:
  - **Actual model execution**

Never size LLM systems CPU-heavy — it bottlenecks, nothing useful.

GPU Memory Hierarchy — Where Attention Bottlenecks

Memory Stack

HBM (GPU DRAM)
↓
L2 Cache
↓
L1 / Shared Memory
↓
Registers

Level	Size	Speed	Role
HBM2e / HBM3	40–80GB	Slowest	Model weights
L2 cache	~50MB	Faster	Reuse data
L1/shared	Small	Very fast	Thread block data
Registers	Tiny	Fastest	Per-thread ops

Attention Bottleneck

Attention requires:

- Reading large matrices (Q, K, V)
- Writing intermediate outputs

👉 **Memory bandwidth becomes bottleneck, not compute**

Arithmetic intensity issue

Attention: `FLOPs / Byte ≈ low`

Meaning: More data movement than compute

👉 → Memory-bound

Flash Attention insight

Optimized attention:

•Avoids writing full QK^T matrix to HBM

•Keeps computation in SRAM (L1/shared memory)

👉 Reduces memory traffic drastically

HPE sizing implication

Critical specs: **HBM bandwidth (TB/s)** , Not just TFLOPS

Example:

• H100: ~3 TB/s bandwidth

👉 For long context models, Memory bandwidth > compute (becomes limiting factor)

HPE Implications

PCIe Gen5 vs NVLink 4.0

👉 **PCIe Gen5**
~128 GB/s (bidirectional per x16 slot)
Used for:
CPU ↔ GPU communication , GPU ↔ storage

👉 **NVLink 4.0 (H100)**
~900 GB/s (GPU ↔ GPU)
👉 **7× faster than PCIe**

👉 **Why this matters**
LLMs often:
Split across multiple GPUs (model parallelism)
Example: 70B model → spans 2–8 GPUs

Communication overhead
During inference GPUs exchange activations
If using PCIe : Bottleneck
If using NVLink : High-speed synchronization

HPE ProLiant Design choices

Architecture	Impact
NVLink-enabled GPU trays	Required for large models
PCIe-only systems	OK for small models
Multi-GPU mesh	Critical for scaling

Rules for sizing

Model Size	Interconnect Needed
< 13B	PCIe OK
13B–70B	NVLink recommended
70B+	NVLink mandatory

CXL Memory Pooling can be impactful

Problem: Context window explosion

Modern models:

•8K → 32K → 128K → 1M tokens

Memory requirement:

•KV cache grows linearly with tokens

KV Cache formula (simplified)

```
Memory ≈ Layers × Heads × Tokens × Head_dim
```

👉 This becomes huge for long context

👉 **GPU memory limitation**

•GPU memory is expensive (HBM)

•Cannot scale infinitely

👉 **CXL (Compute Express Link)**

•Allows:

- Memory pooling across devices
- CPU-accessible expansion memory

CXL Approach

Instead of
Storing KV cache in GPU HBM

We can Offload to:
- CXL memory pool
- Host memory

Storage	Speed	Cost
HBM	Fast	Expensive
CXL memory	Medium	Cheaper
CPU RAM	Slower	Cheapest

HPE implication

Future HPE systems:

👉 CXL-enabled memory pools
- Larger context windows
- Lower cost scaling

👉 Critical for:
- Chatbots with long history
- Document processing
- RAG with large context

Compute Scaling Laws (Chinchilla Optimal)

From DeepMind's Chinchilla paper:
👉 Optimal performance requires balance:

```
Model Size ∝ Training Tokens
```

👉 **Key takeaway**

- Bigger model ≠ always better
- Undertrained large models waste compute

👉 **Chinchilla optimal rule**

For fixed compute:

- Use:
  - Smaller model
  - More training data

Decision	Impact
Very large model	High GPU memory + latency
Smaller optimized model	Better throughput

System sizing implication

👉 **System sizing implication**
Instead of:
Always deploying 70B+
Consider:
7B / 13B models fine-tuned for domain

👉 **Throughput vs Latency tradeoff**

Model	Latency	Throughput
70B	High	Low
13B	Medium	High
7B	Low	Very high

HPE recommendation logic

When sizing systems:

Estimate:

- Tokens/sec requirement
- Concurrent users

Choose:

- Model size based on use case

Then size:

- Number of GPUs
- Memory bandwidth
- Interconnect

HPE recommendation high level flow

When sizing infrastructure:

Step 1: Understand workload
Model size
Context window
Throughput requirement

Step 2: Map to constraints

Constraint	**Driven By**
Compute	GEMM / tensor cores
Memory	KV cache + weights
Bandwidth	Attention
Interconnect	Multi-GPU

HPE recommendation high level flow contd.

Step 3: Hardware mapping

Requirement	Hardware Decision
High GEMM throughput	H100 / tensor cores
Memory heavy	High HBM + CXL
Multi-GPU	NVLink
Cost optimization	Smaller models

Final Key Insights

1. Attention is memory-bound, not compute-bound
2. Tensor cores = everything for LLM performance
3. NVLink > PCIe for scaling
4. Context window drives memory explosion
5. CXL = future of scalable inference
6. Right model size matters more than biggest model